

LANE AND CURVE DETECTION USING DEEP LEARNING

**A PROJECT REPORT
SUBMITTED IN PARTIAL FULFILLMENT OF THE REQUIREMENTS
FOR THE AWARD OF THE DEGREE OF**

**BACHELOR OF ENGINEERING
IN
COMPUTER SCIENCE AND ENGINEERING
(ARTIFICIAL INTELLIGENCE & MACHINE LEARNING)**

Submitted by

Name		Reg. No.
ARUN N	-	201012005
BALAJI G	-	201012007
PRAVEEN N	-	201012028
PUGAZHENTHI K	-	201012032

Project Guide

Dr. S. Saravanan, M.E., Ph.D.,

Assistant Professor

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

ANNAMALAI  **UNIVERSITY**

ANNAMALAI NAGAR – 608002

May 2024

ANNAMALAI



UNIVERSITY

FACULTY OF ENGINEERING AND TECHNOLOGY
DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

This is to certify that the Project report titled “**Lane And Curve Detection Using Deep Learning**” is the bonafide record of the work done by

Name		Reg. No.
ARUN N	-	201012005
BALAJI G	-	201012007
PRAVEEN N	-	201012028
PUGAZHENTHI K	-	201012032

in partial fulfillment of the requirements for the Degree of Bachelor of Engineering in Computer Science and Engineering(Artificial Intelligence & Machine Learning) during the period 2023 - 2024.

Dr. S. SARAVANAN

Assistant Professor
Dept. of Computer Science & Engg.
Project Guide

Dr. R. BHAVANI

Professor & Head
Dept. of Computer Science & Engg.

Internal Examiner

External Examiner

Place: Annamalainagar

Date:

ACKNOWLEDGEMENT

We wish to express our sincere thanks and deep sense of gratitude to **Dr. R. BHAVANI, M.E., Ph.D., Professor & Head**, Department of Computer Science and Engineering, Faculty of Engineering and Technology, Annamalai University for giving us the opportunity to undertake this project.

We would like to convey our heartiest thanks to our project guide **Dr. S. SARAVANAN, M.E., Ph.D., Assistant Professor**, Department of Computer Science and Engineering, for all his help and support. He with his extreme patience has guided us in situation of need for which we are extremely grateful.

We would like to thank our project review committee members **Dr.G. ARULSELVI, M.E., Ph.D., Associate Professor**, **Dr. S. SATHIYA, M.E., Ph.D., Assistant Professor** and **Dr. R. SAMINATHAN, M.E., Ph.D., Associate Professor** Department of Computer Science and Engineering for their great support and encouragement during the course of our project.

We would like to thank all our friends for their support in time of need, encouragement and were always ready to help us without asking for it.

We wish to thank all the technical staff members, incharge of our department laboratories that fulfilled all our project needs and offered us timely help.

Above all, we are indebted to our beloved parents, whose blessings and best wishes have come a long way in making this final year project work a grand success.

ARUN N

BALAJI G

PRAVEEN N

PUGAZHENTHI K

TABLE OF CONTENT

Chapter No.	Content	Page No.
	ABSTRACT(ENGLISH)	i.
	ABSTRACT(TAMIL)	ii.
	LIST OF FIGURES	iii.
1.	INTRODUCTION	1
	1.1 OBJECTIVE	2
	1.2 FEATURES OF LANE MARKING	
	1.3 TYPES OF ROADLINES	3
	1.4 VISION BASED LANE DETECTION	4
	1.5 DEEP LEARNING BASED LANE DETECTION	6
	1.6 ORGANIZATION OF THE THESIS	
2.	LITERATURE SURVEY	7
3.	PROPOSED METHODOLOGY	10
	3.1 CONVOLUTIONAL LSTM	10
	3.2 LSTM ARCHITECTURE	13
	3.3 SYSTEM REQUIREMENTS	19
	3.4 SOFTWARE DESCRIPTION	19

4.	SYSTEM IMPLEMENTATION	25
	4.1 LSTM and ConvLSTM FOR TIME SERIES DATA	27
	4.2 MODULES	31
5	EXPERIMENTAL PROCESS	33
	5.1 DATASET	33
	5.2 SCREENSHOTS	34
	5.3 PERFOMANCE MEASURES	36
6	CONCLUSION AND FUTURE WORK	39
	REFERENCES	40

ABSTRACT

For autonomous cars and sophisticated driver assistance systems, lane detection in driving scenarios is a crucial module. Many advanced techniques for lane detecting have been presented in the past few years. Nevertheless, most approaches concentrate on lane detection from a single image, which frequently results in subpar handling of certain particularly problematic scenarios like intense shadow, severe mark degradation, significant vehicle occlusion, and so forth. The continuous line formations on a road are actually called lanes. Thus, by combining information from earlier frames, it may be possible to deduce the lane that cannot be reliably recognised in a single frame. Using many frames from a continuous driving environment, we examine lane detection in order to achieve this goal. We then propose a hybrid deep architecture that combines the Convolutional Neural Network (CNN) with the Recurrent Neural Network (RNN). In other words, each frame's information is abstracted by a CNN block, which then feeds the CNN features of several continuous frames, which have the property of a time series into the RNN block for lane prediction and feature learning. The suggested approach performs better in lane detection than the competing methods, especially when managing challenging scenarios, according to extensive tests conducted on two large-scale datasets.

2019年，中国人工智能产业规模达到1500亿元，同比增长30%。其中，计算机视觉、自然语言处理和语音识别三大领域表现最为突出。百度、阿里巴巴和腾讯等企业纷纷加大研发投入，推动人工智能技术在医疗、金融、教育等领域的落地应用。

在计算机视觉领域，百度推出的AI开放平台吸引了大量开发者，推动了人脸识别、图像识别等技术的普及。阿里巴巴的达摩院则在自动驾驶和机器人领域取得了突破。腾讯则在游戏和社交领域广泛应用人工智能技术，提升了用户体验。

自然语言处理方面，百度的智能客服系统已经覆盖了大部分用户，大大提高了服务效率。阿里巴巴的AI助手则在电商领域发挥了重要作用，帮助商家更好地了解用户需求。腾讯的AI技术在游戏NPC和社交推荐方面也有广泛应用。

语音识别领域，百度的语音助手小度已经进入了千家万户，成为用户日常生活中不可或缺的一部分。阿里巴巴的AI技术在智能音箱和车载系统领域也有广泛应用。腾讯则在语音识别和合成技术方面取得了显著进展，为游戏和社交应用提供了有力支持。

除了三大领域，人工智能在医疗、金融、教育等行业的应用也在不断深入。例如，在医疗领域，AI辅助诊断系统已经能够帮助医生更准确地识别疾病。在金融领域，AI风控系统能够有效识别和防范金融风险。在教育领域，AI个性化学习系统可以根据学生的学习情况提供定制化的学习方案。

然而，人工智能的发展也面临着一些挑战。首先是数据隐私和安全问题，随着人工智能技术的广泛应用，大量的个人数据被收集和存储，如何确保这些数据的安全和隐私成为了一个亟待解决的问题。其次是人才短缺问题，人工智能领域的高端人才非常稀缺，企业需要投入大量的资源进行人才培养。最后是伦理和监管问题，人工智能技术的应用可能会带来一些伦理上的争议，需要建立健全的监管体系来规范其发展。

展望未来，人工智能将继续保持高速发展的态势。随着技术的不断进步和应用的不断拓展，人工智能将在更多领域发挥重要作用，为人类社会带来更多的便利和福祉。同时，我们也需要关注人工智能发展中的潜在风险，加强监管和伦理建设，确保人工智能技术健康、可持续发展。

LIST OF FIGURES

Fig.No	Title	Page No.
1.1	Lane detection block diagram	4
3.1	Architecture of the proposed work	11
3.2	Convolution Network	13
3.3	Recurrent Neural Network	13
3.4	LSTM memory cell	16
3.5	Detailed Structure of LSTM	17
4.1	Architecture of ConvLSTM	25
4.2	A typical urban road driving scenario	28
4.3	ConvLSTM-based model framework	30
5.1	Training images and test images from TuSimple dataset	33
5.2	Main GUI window for the project	34
5.3	Select the video file	34
5.4	Selected video input	35
5.5	Lane and Curve Detection	35
5.6	Lane and Curve Detection	36

CHAPTER – 1

INTRODUCTION

During vehicle driving the coordination of our vision, hands, legs, mind and our perception plays an important role to take accurate real time decisions to drive safely on the road. During driving session your sensors like eyes, ears and perception senses the environmental variables like distance between surrounding vehicles, their speed, safely vehicle overtaking needs etc. These sensed environmental variable evaluates in our mind and accordingly we taking driving decisions. However, our driving and required decisions gradually improve with the increasing driving experience. The person who has just started driving feels difficulty to take appropriate decisions while driving because of less experience and an-adequate perception. As driving experience in-creases we learn from the real time environmental situations and continuously refine driving time decisions.

Auto driving vehicle is one of the revolutionary and raising research problem where computer vision can do a remarkable contribution. In this decade both traditional and deep learning models have remarkably contributed in developing various autonomous driving vehicle algorithms. In computer vision, information extraction from the real time driving scenarios through patterns identification, recognition, and object detection happen and these environmental variables helps the algorithm to generate automatic guidance to the driving and controlling system. Such algorithm comes under the category of self-learning algorithms, which refine the decisions by learning through increasing driving experience. Lane detection during continuous driving plays a crucial role autonomous driving decision taking. However, lane detection itself has various challenges i.e. fading and removal of road sector lines, tree shadow, dark on the road, street light etc. However, various researchers are working over the autonomous driving vehicles and with the evaluation of computer vision robust algorithms automatic lane detection during driving has been a possible scenario.

Lane detection itself has some sub-processes like identification of Region Of Interest (ROI) and its segmentation to have the boundary features of the lane. Traditionally intensity change, gradient features and hand crafted features have been used for lane detection. However, these traditional lane detection techniques generate comfortable accuracy in lane detection on the high ways on the other hand they straggle when lanes are not marke

properly or they have been faded or removed and even with unfavorable weather conditions like rain etc.

Generally, road lanes are line structures and continuous on the pavement, appeared either in solid or dash. As the driving scenes are continuous and are largely overlapped between two neighboring frames, the position of lanes in the neighboring frames is highly related. More precisely, the lane in the current frame can be predicted by using multiple previous frames, even though the lane may suffer from damage or degradation brought by shadows, stains and occlusion. This motivates us to system lane detection by using images of a continuous driving scene.

For the lane detection task, the following current challenges exist:

1. Lanes are often not complete. The occlusion of other vehicles and natural wear cause the lanes to be incomplete most of the time, which greatly increases the difficulty of detection.
2. The detection of lanes will be affected by the environment. Different seasons, weather and time will lead to differences in the collected road information, resulting in lower recognition. Rain, dust, or snow on the camera may change the perspective and viewing range of the camera, causing errors in the result.
3. There is a lack of a unified benchmark dataset of suitable difficulty to more accurately evaluate the performance of each method.
4. Few models pay attention to the fact that the lanes are continuous. The vast majority of methods focus on the detection of a single image rather than continuous images. This results in very important sequence information being almost wasted.

1.1 OBJECTIVE

The aim of this project is to avoid accidental deaths and provide a better safety on roads, by use of advanced technologies in driving assistances system. The clear idea of the important aims and objectives of this project is explained with the help of block diagram.

1.2 FEATURES OF LANE MARKING

- Adapts to various types of roads.

- Color, style and width of markings recognition.
- Detects all road markings in the picture.
- Integrated navigation system, see the track ego lane change and offer advice.
- Adapts to different weather and light condition.

1.3 TYPES OF ROAD LINES

- Continuous centre lines

You can cross a continue center line to enter or leave a road, but cannot overtake.

- Broken center lines

You are allowed to overtake across a broken centre line or broken centre line.

- Continuous edges lines

Boundary lines (edges lines) are used to select the edge of the road. The area to the left edge of the line is the axis of the road which is also called shoulder of the road. This is not just an extra lane for vehicles to travel in. But, cyclists may also travel on the shoulder road. Vehicle also used the road edges lines in case when vehicle entering or leaving the road, stopping at the side of a road, turning at an intersection etc.

1.4. VISION BASED LANE DETECTION

Real time vision-based lane detection method is presented to find the position and type of lanes in each video frame proposed a method for lane detection effective combination of filter functions edge-Link channels. The first filter means candidates are sought in the Region Of Interest (ROI). During the project, a broad edge linking algorithm circuit slot marginal land used to produce the filter width for wider access board and serves as a way to project the edge orientation and tape are used to filter the channels marked border pair link candidates. A linear model based method has been developed for detecting the tracking markers in real time.

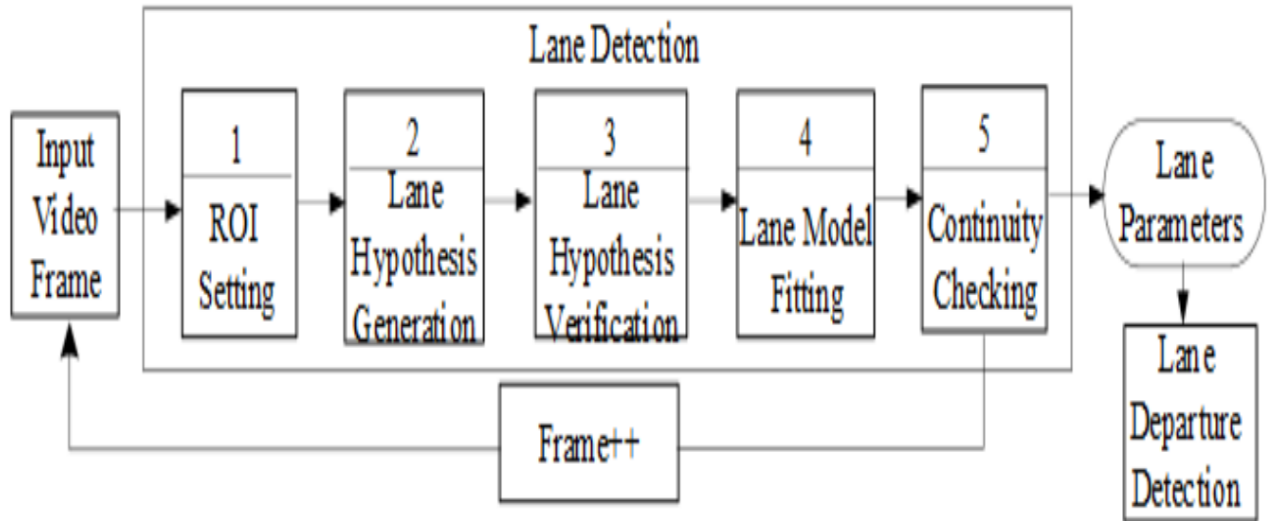


Fig. 1.1 Lane detection block diagram

To estimate the linear model is robust filtering capabilities such as efficient roads and edges, color, width and direction are combined to follow the markings on the parameters of the linear model is used to represent traces are calculated. Lane position can be determined from the linear model parameters and Lane Departure can be calculated. A diagram of the overall management of the Lane Departure proposed method of detection is shown in Fig1.1. Image characteristics of the lane that comes with a new method for lane detection and tracking can accurately extract by contrast gray image and processing binary .the filter strengthening increasingly application track binary information. Smooth Gaussian image Canny operator, processed for the detection of channel outlines, when the corner detection method is used for the image coordinates of the corners, finally this method is used to get the optimized lane step by step, the lane parameters used to obtain more accurate track of and extraction of the curve is more perfect. The method not only improves the accuracy of path discovery, but also ensures the safety of the vehicle.

1.5 DEEP LEARNING BASED LANE DETECTION

Deep learning-based lane detection is a computer vision technique that employs deep neural networks to detect lane boundaries on roads from images or videos. It's a critical component in various applications like autonomous driving, Advanced Driver-Assistance Systems (ADAS), and robotics. Here's a general outline of how it works:

1. **Dataset Collection:** Deep learning models require large datasets for training. In the case of lane detection, this involves collecting images or videos of roads with annotated lane markings.
2. **Data Preprocessing:** Before feeding the data into the model, preprocessing steps like resizing, normalization, and data augmentation may be applied to improve model performance and generalization.
3. **Model Architecture:** Convolutional Neural Networks (CNNs) are commonly used for lane detection tasks. Architectures like U-Net, VGGNet, or custom-designed networks are popular choices. These networks are designed to learn hierarchical features from input images, enabling them to recognize lane markings.
4. **Training:** The model is trained on the annotated dataset using techniques like backpropagation and optimization algorithms such as Stochastic Gradient Descent (SGD) or Adam. The objective is to minimize a loss function that measures the difference between predicted lane boundaries and ground truth annotations.
5. **Post-processing:** After obtaining predictions from the model, post-processing techniques may be applied to refine the results. This could involve methods like morphological operations, polynomial fitting, or probabilistic models to smooth and refine the detected lane boundaries.
6. **Evaluation:** The trained model is evaluated on a separate validation dataset to assess its performance. Metrics like accuracy, precision, recall, and Intersection over Union (IoU) are commonly used to quantify the model's performance.
7. **Deployment:** Once the model meets the desired performance criteria, it can be deployed in real-world applications. This involves integrating it into the larger system architecture, such as an autonomous vehicle or an ADAS system.

It's worth noting that deep learning-based lane detection is a rapidly evolving field, with ongoing research aimed at improving accuracy, robustness, and efficiency. Techniques such as semantic segmentation, instance segmentation, and multi-task learning are being explored

to enhance the capabilities of lane detection systems. Additionally, the availability of large-scale datasets and advancements in hardware (e.g., GPUs, TPUs) continue to drive progress in this area.

1.6 ORGANIZATION OF THE THESIS

The report is organized as follows:

Chapter 2: This chapter deals with Literature Survey which are used in recent researches.

Chapter 3: This chapter discusses about the Hardware and software specification which is used for the proposed work.

Chapter 4: This chapter describes the proposed methodology and its algorithm,

Chapter 5: This chapter describes about the system design & module implementations

Chapter 6 : This chapter describes the experimental process with the dataset information and the experimental result, screenshots and analysis.

Chapter 7: This chapter describes about Conclusion and future enhancement.

CHAPTER – 2

LITERATURE SURVEY

A literature survey is an essential component of any research project, providing a comprehensive overview of existing knowledge and previous studies on a specific topic. This survey is conducted to understand the scope of current research, identify gaps in the literature, and set the context for the research question at hand. By systematically analyzing academic articles, books, conference papers, and other scholarly sources, a literature survey allows researchers to track the evolution of theories and concepts, evaluate methodologies, and compare findings from different studies.

Zillur Rahman et al., proposes a CNN based approach for improved lane detection and classification, addressing existing model limitations in challenging scenarios. The authors curated a comprehensive dataset, fine-tuned state-of-the-art models, and introduced a lightweight lane classifier, enhancing accuracy in lane localization. Evaluations on TuSimple, Caltech, and LVLane datasets show superior performance in challenging scenarios and real-time lane type classification, with a focus on implications for autonomous vehicles and ADAS systems. It gives accuracy 97.9% [1].

Qin Zou et al., addresses the limitations of single-image lane detection methods in challenging driving scenarios, proposing a hybrid deep architecture combining Convolutional Neural Networks (CNN) and Recurrent Neural Networks (RNN). By utilizing information from multiple continuous frames, the model outperforms competitors, especially in difficult situations. The proposed architecture, compared to baseline models using single images, demonstrates significantly better results, showcasing the effectiveness of incorporating continuous frames. ConvLSTM is found superior to FcLSTM in sequential feature learning, resulting in higher precision, recall, and accuracy values. The models exhibit robustness in diverse driving scenarios and effectively avoid false recognitions [2].

Manoj et al., explores the application of RNN and CNN in road lane detection for autonomous driving vehicles, using a dataset of 25,000 input samples and 2,500 test images. CNN outperforms RNN with a 97.91% accuracy, crucial for continuous driving decisions. Challenges include fading lines, tree shadows, and low-light conditions. The proposed framework is evaluated on both CNN and RNN using a self-created dataset and BDD100K,

showcasing CNN's superior performance but acknowledging areas for improvement in handling curved lanes and shadows [3].

Rustam et al., introduces a novel approach to road lane detection for autonomous vehicles, utilizing the Mask R-CNN model. The proposed method, enhanced with a diverse custom dataset and a multi-scale feature extraction technique, outperforms existing techniques in accuracy, processing speed, and adaptability. Extensive experiments validate the high precision and recall rates of the Mask R-CNN-based approach, making it an ideal solution for real-time implementation in autonomous vehicles for safe and efficient navigation [4].

Fareha et al., focuses on implementing a computer vision algorithm for lane lines detection on a California highway using real data from a vehicle's front-facing camera. The processed video highlights lane lines, enabling calculation of the vehicle's offset for steering control. The method utilizes the Hough line transform to detect and draw lanes, ultimately creating a blended image for improved lane visualization [5].

Raviteja et al., addresses the challenges in predicting images for highway investigation within the Intelligent Transportation System (ITS). It introduces a novel approach using a Deep-Convolutional Neural Network (D-CNN) for sample-level classification, integrating uncertainty evaluation to enhance robustness. The proposed D-RNN then analyzes feature representation from sample bags, achieving a superior trade-off in classification accuracy (92.9%), sensitivity (76.40%), and specificity (100%) compared to other methods, demonstrating its efficacy in handling changing viewpoints and diverse conditions [6].

Zhang et al., proposed is a two-stage YOLO v3-based adaptive lane feature learning algorithm, modifying structural parameters for lane detection. An automatic method generates label images for first-stage network training in simple scenarios. An adaptive edge detection algorithm and shielding of unrecognized lanes in complex scenarios enable training of a second-stage model, achieving high accuracy (96.2%) on KITTI and Caltech datasets [7].

Wang et al., proposes a dual model using Convolutional neural networks for improved lane detection in varying scenarios, combining edge-cloud computing to enhance data processing efficiency. It introduces a custom neural network for dynamic inverse perspective transformation, addressing ground changes and achieving better model performance in

challenging conditions like insufficient lighting and lane degradation. The method outperforms traditional and deep learning-based lane detection models, showcasing superior recognition efficiency 97.45% [8].

Abdullah et al., Introduces encode-decode instant segmentation network (EDIS-Net), a Deep Learning methodology for Lane Marking Detection (LMD) in diverse environments, addressing challenges like light variations and obstacles. It is based on E-Net architecture, it utilizes binary and instant segmentation for accurate lane pixel information extraction, achieving 97.39% accuracy on Tusimple dataset and strong performance on CalTech and local datasets. The proposed EDIS-Net demonstrates promising results, showcasing its effectiveness in confident lane marking recognition across various scenarios [9].

Khan et al., introduces LLDNet, a lightweight CNN model for lane detection, addressing challenges of real-world road variations and adverse weather conditions. Integrated with channel and spatial attention modules, it outperforms state-of-the-art architectures in terms of accuracy, measured by dice coefficient and IoU. It achieves 94.2% accuracy. Successfully tested on diverse road scenarios, LLDNet offers a practical and efficient solution for lane detection, suitable for implementation in embedded devices [10].

CHAPTER 3

PROPOSED METHODOLOGY

Lane detection using Convolutional Neural Networks (CNNs) and Long Short-Term Memory (LSTM) is a technique used in autonomous driving systems. CNNs are proficient at extracting spatial features from individual frames, LSTM networks are useful for modeling temporal dependencies

and capturing long-range context in sequential data. By combining these two types of neural networks, the model can effectively leverage both spatial and temporal information for accurate lane detection in video streams.

- This proposed system addresses the limitations of single-image lane detection methods in challenging driving scenarios, proposing a hybrid deep architecture combining Convolutional Neural Networks (CNN) and Recurrent Neural Networks (RNN).
- By utilizing information from multiple continuous frames, the model outperforms competitors, especially in difficult situations.
- ConvLSTM is found superior to LSTM in sequential feature learning.

3.1 CONVOLUTIONAL LSTM

Modeling the multiple continuous frames of driving scene as time-series, the RNN block in the proposed network accepts feature map extracted on each frame by the encoder CNN as input. To handle various time-series data, different kinds of RNN have been proposed, e.g., LSTM and GRU. In this work, an LSTM network is employed, which generally outperforms the traditional RNN model with its ability in forgetting unimportant information and remembering essential features, by using cells in the network to judge whether the information is important or not. A double-layer LSTM is applied, with one layer for sequential feature extraction and the other for integration. The traditional full-connection LSTM is time- and computation- consuming. Therefore, we utilize the convolutional LSTM (ConvLSTM) in the proposed network.

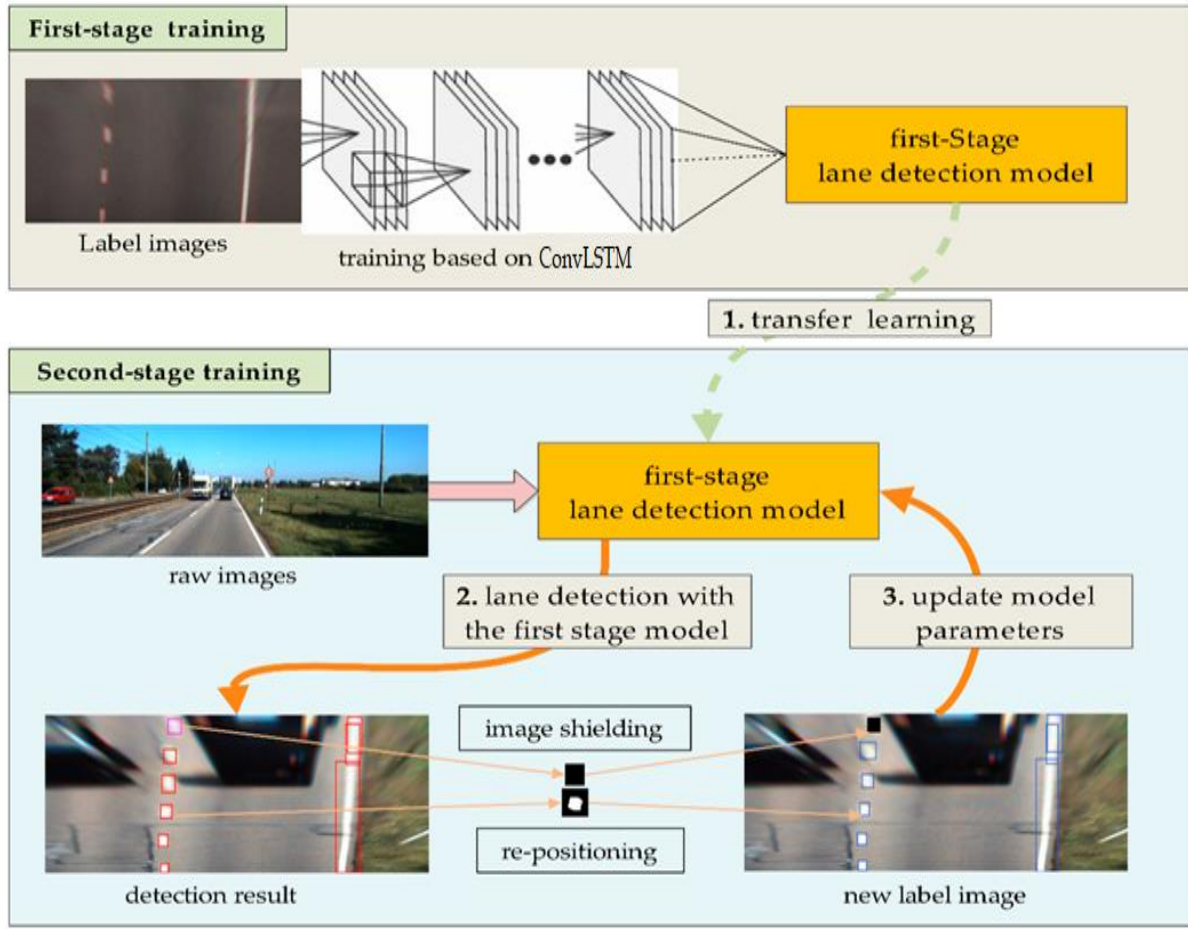


Fig. 3.1. Architecture of the proposed work

LSTM is a basic building block in deep learning, which is talent in processing temporal information. Convolutional LSTM (ConvLSTM) is a kind of LSTM equipped with convolution operations, and has been widely used in video analysis due to their feedback mechanism on temporal dynamics and the abstraction power on image representation. There are different ways to use ConvLSTM as a basic building block. For semantic video segmentation specifically, ConvLSTMs are popularly used for efficiently processing the two-dimensional time-series inputs. The video segmentation task was transformed into a regression problem, where a ConvLSTM was set between the convolutional encoder-decoder to predict every frame. A convolutional GRU was introduced to fuse the two-stream representation obtained by a normal convolutional appearance network and an optical-flow motion network. A different structure was presented, which placed a two-stream ConvLSTM after the encoder to get spatial and temporal region proposals. A spatio-temporal saliency learning module was constructed by placing a pyramid dilated bidirectional ConvLSTM after the spatial saliency learning module.

3.1.1 Integrating CNN with LSTM:

For seamlessly integrating the DRNN with DCNN, a novel fusion strategy is presented, where the DCNN consists of an encoder and a decoder with fully-convolution layers, and the DRNN is implemented as an LSTM network. The DCNN abstracts each frame into a low dimension feature map

A CNN belongs to the family of deep learning algorithms family which is quite good with image classification and NPL problems solving. In this work CNN is used for image classification for the input image size (200, 200). In the first layer consists of 16 neurons followed by a Maxpool2D layer of (2,2). The second layer consists of 32 neurons followed by a Maxpool2D layer of (2,2) and third layer consists of 64 neurons followed by a Maxpool2D layer of (2,2). Above three layers are then flattened and then passed into a Dense layer of 512 neurons. Finally, for the image classification in three different categories (i.e. “Left”, “Right” and “Straight”) a dense layer with 3 neurons is used. The proposed CNN model is trained over the 17,359,907 parameters.

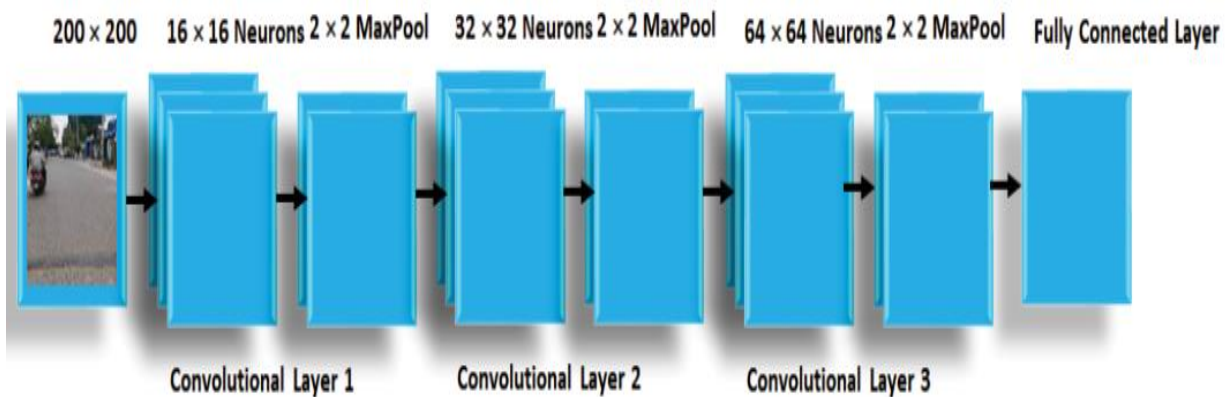


Fig. 3.2. Convolution Network

Another experiment setup is created using RNN algorithm. Alike CNN architecture, RNN algorithm is also best perform in Natural Language Processing and image process. The first layer of the proposed RNN is combined with LSTM which has 128 neurons and a dropout-layer at 0.2 value. Next RNN layer is combined with LSTM with again 128 neurons and the activation function used is ‘Relu’ followed by a dropout-layer with 0.2 value. The next subsequent layer is Dense-layer with 32-neurons, “Relu” activation function with a subsequent Dropout-layer. Finally, dense layer along the 3-classification neurons is created.

3.2. LSTM ARCHITECTURE

The idea behind RNNs is to make use of sequential information. To predict the next data in a series, it's important to know which data came before it. RNNs are called recurrent because they perform the same task for every element of a sequence, with the output being depended on the previous computations and already know that they have a “memory” which captures information about what has been calculated so far.

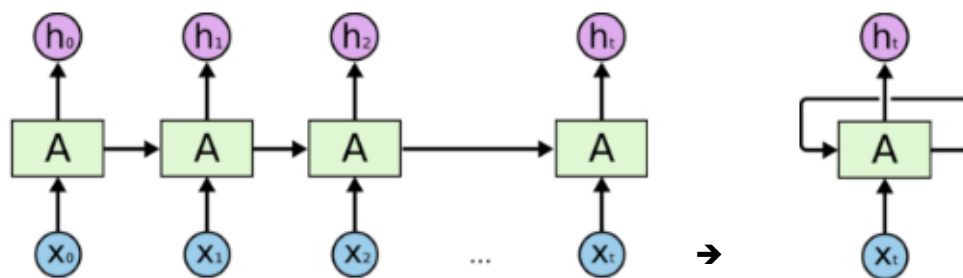


Fig. 3.3 Recurrent Neural Network

A **Recurrent Neural Network (RNN)** is basically a standard neural network that has been extended across time by having edges that feed into the next time step instead of into the next layer in the same time step. RNN is designed to recognize sequences, for example, a speech signal or a text. It has a cycle inside that implies the presence of short memory in the net.

Different types of RNN's

The core reason is recurrent network allows operating over sequences of vectors: Sequences in the input, the output, or in the most general case both. The RNN has various formations, such as,

One-to-one:

This also called as Plain Neural networks. It deals with Fixed size of input to Fixed size of Output where they are independent of previous information/output.

One-to-Many:

it deals with fixed size of information as input that gives sequence of data as output.

Many-to-One:

It takes Sequence of information as input and outputs a fixed size of output.

Bidirectional Many-to-Many:

Synced sequence input and output. Notice that in every case are no pre-specified constraints on the lengths sequences because the recurrent transformation (green) is fixed and can be applied as many times as we like.

Deep view into RNN's:

In a simple Neural Network, you can see Input unit, hidden units and output units that process information independently having no relation to previous one. Also, here we gave different weights and bias to the hidden units giving no chance to memorize any information. Where Hidden layer in RNN's have same weights and bias throughout the process giving them the chance to memorize information processed through them.

Current time stamp:

look at the above figure, where the formula for the Current state:

$$h_t = f(h_{t-1}, x_t) \quad (1)$$

function using previous times stamp and input gives current time stamp

where $\mathbf{O}_t$ is output state, $\mathbf{h}_t \rightarrow$ current time stamp, $\mathbf{h}_{t-1} \rightarrow$ is previous time stamp, and $\mathbf{x}_t$ is passed as input state.

Applying activation function:

$$h_t = \tanh(w_{hh}h_{t-1} + w_{xh}x_t) \quad (2)$$

Activation function

$\mathbf{W}$ is weight, $\mathbf{h}$ is the single hidden vector, $\mathbf{W}_{hh}$ is the weight at the previous hidden state, $\mathbf{W}_{hx}$ is the weight at the current input state. Where **tanh** is the Activation function, that implements a Non-linearity that squashes the activations to the range[-1, 1]

Output:

$$y_t = w_{hy} h_t \quad (3)$$

Where, Y_t is the output state. W_{hy} is the weight at the output state.

Long Short-Term Memory (LSTM):

LSTM pioneered by Schmidhuber and Hochreiter in 1997. It is a kind of RNN which is capable of learning long-term dependencies. LSTM networks are specially designed to remember the information for long periods to avoid the long-term dependency problem.

Long short-term memory (LSTM) is an artificial recurrent neural network (RNN) architecture used in the field of deep learning. Unlike standard feedforward neural networks, LSTM has feedback connections. It can process not only single data points (such as images), but also entire sequences of data (such as speech or video).

The long short-term memory (LSTM) contains distinct units called memory unit in the recurrent hidden layer. Every memory unit has 3 gates named as **Forget gate**, **Input gate**, **Output gate**. An input gate which determines whether or not to let the new input in, a forget gate which deletes information that is not important and an output gate which decides what information to output.

The input gate controls the flow of input activations to the activations into the rest of the network. The forget gate scales the internal state of the cell before adding it as input to the cell through the self-recurrent connection of the cell, therefore adaptively forgetting or resetting the cell's memory.

Forget Gate: This gate Decides which information to be omitted in from the cell in that particular time stamp. It is decided by the **sigmoid function**. it looks at the previous state(h_{t-1}) and the content input(X_t) and outputs a number between **0**(omit this)and **1**(keep this)for each number in the cell state C_{t-1} .

$$f_t = \sigma(w_f \cdot [h_{t-1}, x_t] + b_f) \quad (4)$$

Update Gate/input gate: It decides how much of this unit is added to the current state.

$$i_t = \sigma(w_i \cdot [h_{t-1}, x_t] + b_i) \quad (5)$$

$$C_t = \tanh(w_c \cdot [h_{t-1}, x_t] + b_c) \quad (6)$$

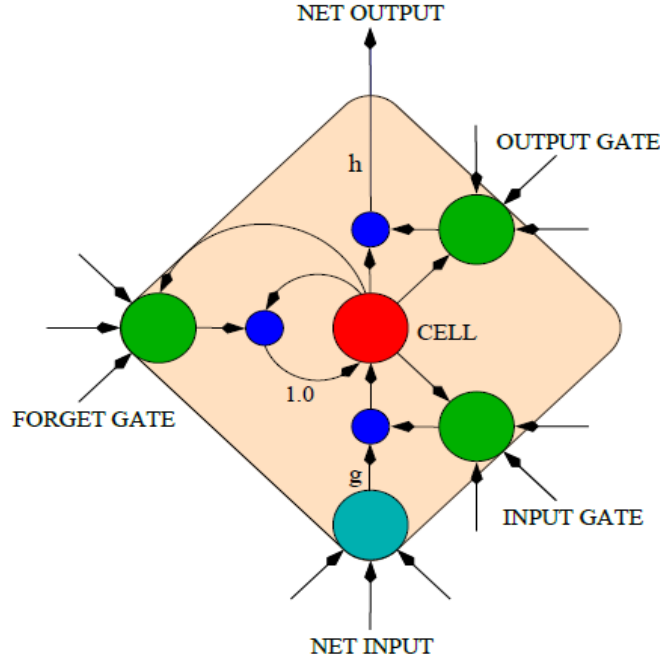


Fig. 3.4 LSTM memory cell

Sigmoid function decides which values to let through **0,1**. and **tanh** function gives weightage to the values which are passed deciding their level of importance ranging from **-1** to **1**.

Output Gate: It decides which part of the current cell makes it to the output.

$$o_t = \sigma(w_o \cdot [h_{t-1}, x_t] + b_o) \quad (7)$$

$$h_t = O_t * \tanh(c_t) \quad (8)$$

Sigmoid function decides which values to let through 0,1. and tanh function gives weightage to the values which are passed deciding their level of importance ranging from -1 to 1 and multiplied with output of Sigmoid.

These three gates are analog gates based on the sigmoid function which works on the range 0 to 1. These three sigmoid gates can be seen in Fig. 3.4 below. A horizontal line that can be seen running through the cell represents the cell state.

In LSTM network, two states are being transferred to the next cell, the cell state, and the hidden state. The cell state is the main chain of data flow, which allows the data to flow forward essentially unchanged. However, some linear transformations may occur. The data

can be added to or removed from the cell state via sigmoid gates. A gate is similar to a layer or a series of matrix operations, which contain different individual weights. LSTMs are designed to avoid the long-term dependency problem because it uses gates to control the memorizing process.

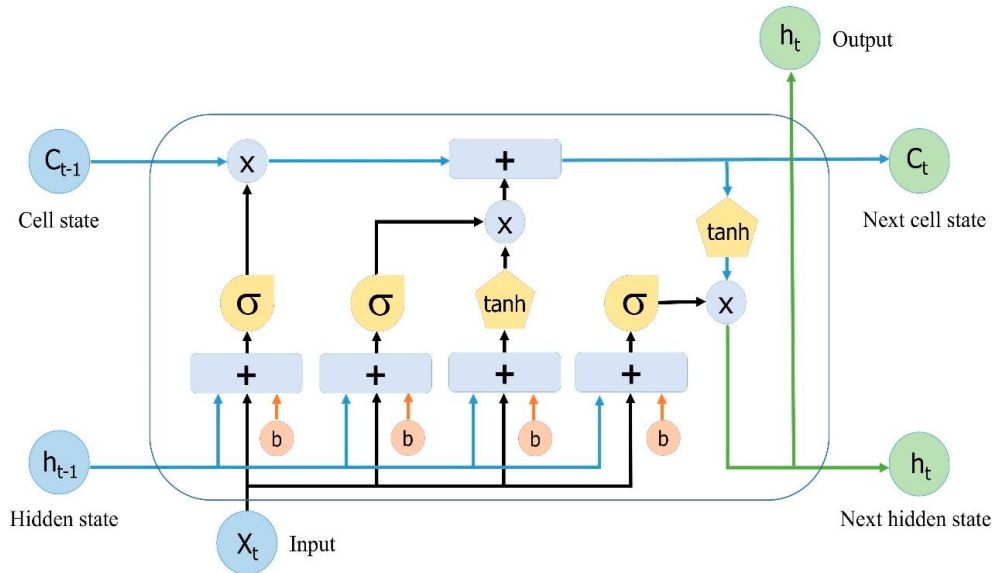


Fig.3.5. Detailed Structure of LSTM

The first step in constructing an LSTM network is to identify information that is not required and will be omitted from the cell in that step. This process of identifying and excluding data is decided by the sigmoid function, which takes the output of the last LSTM unit (h_{t-1}) at time $t - 1$ and the current input (X_t) at time t . Additionally, the sigmoid function determines which part from the old output should be eliminated. This gate is called the forget gate (or f_t); where f_t is a vector with values ranging from 0 to 1, corresponding to each number in the cell state, C_{t-1} .

Table 3.1 LSTM Description

Inputs	Outputs	Nonlinearities	Vector operations
X_t – Current Input	C_t – New updated memory	σ –Sigmoid layer	$\times$ – Scaling of information
C_{t-1} –Memory from last LSTM unit	h_t – current output	$\tanh$ – Tanh layer	$+$ – Adding information

$$f_t = \sigma(W_f [h_{t-1}, X_t] + b_f). \quad (8)$$

Herein, σ is the sigmoid function, and W_f and b_f are the weight matrices and bias, respectively, of the forget gate. The following step is deciding and storing information from the new input (X_t) in the cell state as well as to update the cell state. This step contains two parts, the sigmoid layer and second the tanh layer. First, the sigmoid layer decides whether the new information should be updated or ignored (0 or 1), and second, the tanh function gives weight to the values which passed by, deciding their level of importance (-1 to 1). The two values are multiplied to update the new cell state. This new memory is then added to old memory C_{t-1} resulting in C_t .

$$i_t = \sigma(W_i [h_{t-1}, X_t] + b_i), \quad (9)$$

$$N_t = \tanh(W_n [h_{t-1}, X_t] + b_n), \quad (10)$$

$$C_t = C_{t-1}f_t + N_t i_t. \quad (11)$$

Here, C_{t-1} and C_t are the cell states at time $t-1$ and t , while W and b are the weight matrices and bias, respectively, of the cell state.

In the final step, the output values (h_t) is based on the output cell state (O_t) but is a filtered version. First, a sigmoid layer decides which parts of the cell state make it to the output. Next, the output of the sigmoid gate (O_t) is multiplied by the new values created by the tanh layer from the cell state (C_t), with a value ranging between -1 and 1.

$$O_t = \sigma(W_o [h_{t-1}, X_t] + b_o), \quad (12)$$

$$h_t = O_t \tanh(C_t). \quad (13)$$

Here, W_o and b_o are the weight matrices and bias, respectively, of the output gate.

3.3. SYSTEM REQUIREMENTS

3.3.1 HARDWARE REQUIREMENTS

Processor	:	Intel dual core processor
RAM	:	4 GB
Hard disk	:	500 GB
Processor Speed	:	3.12 GHz
Monitor	:	21' Samsung LCD
Keyboard	:	108 mercury keyboard
Mouse	:	Logitech mouse

3.3.2 SOFTWARE REQUIREMENTS

Language	:	Python 3.6
Tools Used	:	Spyder 5
Operating System	:	Windows 10

3.4 SOFTWARE DESCRIPTION

PYTHON

Python is an interpreter, object-oriented, high-level programming language with dynamic semantics. Its high-level built-in data structures, combined with dynamic typing and dynamic binding, make it very attractive for Rapid Application Development, as well as for use as a scripting or glue language to connect existing components together. Python's simple, easy to learn syntax emphasizes readability and therefore reduces the cost of program maintenance. Python supports modules and packages, which encourages program modularity and code reuse. The Python interpreter and the extensive standard library are available in source or binary form without charge for all major platforms and can be freely distributed.

Python supports multiple programming paradigms, including object-oriented, imperative, and functional programming or procedural styles. It features a dynamic type system and automatic memory management and has a large and comprehensive standard library. Python interpreters are available for many operating systems, allowing Python code to run on a wide variety of systems. Using third-party tools, such as Py2exe or Pyinstaller, Python code can be packaged into stand-alone executable programs for some of the most popular operating systems, so Python-based software can be distributed to, and used on, those environments with no need to install a Python interpreter.

LIBRARIES IN PYTHON

Python has a large standard library, commonly cited as one of Python's greatest strengths, providing tools suited to many tasks. This is deliberate and has been described as a "batteries included" Python philosophy. For Internet-facing applications, many standard formats and protocols (such as MIME and HTTP) are supported. Modules for creating graphical user interfaces, connecting to relational databases, pseudorandom number generators, arithmetic with arbitrary precision decimals, manipulating regular expressions, and doing unit testing are also included.

Some parts of the standard library are covered by specifications, but most modules are not. They are specified by their code, internal documentation, and test suite (if supplied). However, because most of the standard library is cross-platform Python code, only a few modules need altering or rewriting for variant implementations. Software for Python, contains over 86,000 packages offering a wide range of functionality, including:

- Graphical user interfaces, web frameworks, multimedia, databases, networking and communications
- Test frameworks, automation and web scraping, documentation tools, system administration
- Scientific computing, text processing, image processing

IMPLEMENTATIONS

The main Python implementation, named CPython, is written in C meeting the C89 standard. It compiles Python programs into intermediate bytecode, which is executed by the virtual machine. CPython is distributed with a large standard library written in a mixture of C

and Python. It is available in versions for many platforms, including Windows and most modern Unix-like systems. CPython was intended from almost its very conception to be cross-platform.

- PyPy is a fast, compliant interpreter of Python 2.7 and 3.2. Its just-in-time compiler brings a significant speed improvement over CPython. A version taking advantage of multi-core processors using software transactional memory is being created.
- Stackless Python is a significant fork of CPython that implements microthreads; it does not use the C memory stack, thus allowing massively concurrent programs. PyPy also has a stackless version.
- MicroPython is a lean, fast Python 3 variant that is optimised to run on microcontrollers.

There are several compilers to high-level object languages, with either unrestricted Python, a restricted subset of Python, or a language similar to Python as the source language:

- Jython compiles into Java byte code, which can then be executed by every Java virtual machine implementation. This also enables the use of Java class library functions from the Python program.
- IronPython follows a similar approach in order to run Python programs on the .NET Common Language Runtime.
- The RPython language can be compiled to C, Java bytecode, or Common Intermediate Language, and is used to build the PyPy interpreter of Python.
- Pyjamas compiles Python to JavaScript.
- Shed Skin compiles Python to C++.
- Cython and Pyrex compile to C.

The software requirements are description of features and functionalities of the target system. Requirements convey the expectations of users from the software product. The requirements can be obvious or hidden, known or unknown, expected or unexpected from client's point of view.

1. Jupyter Notebook

The Jupyter Notebook is an open source web application that you can use to create and share documents that contain live code, equations, visualizations, and text. Jupyter ships with the IPython kernel, which allows you to write your programs in Python, but there are currently over 100 other kernels that you can also use. The Jupyter Notebook combines three components:

- **The notebook web application:** An interactive web application for writing and running code interactively and authoring notebook documents.
- **Kernels:** Separate processes started by the notebook web application that runs users' code in a given language and returns output back to the notebook web application. The kernel also handles things like computations for interactive widgets, tab completion and introspection.
- **Notebook documents:** Self- contained documents that contain a representation of all content visible in the notebook web application, including inputs and outputs of the computations, narrative text, equations, images, and rich media representations of objects. Each notebook document has its own kernel.

2. Spyder

Spyder (Scientific Python Development Environment, previously known as Pydee) is a simple and lightweight, yet powerful interactive development environment for scientific programming in the Python language. This software is open source and cross-platform. It can be installed on Windows using Python(x,y), WinPython or Anaconda, on Mac OS via Anaconda or MacPorts. Spyder also can be integrated to widely used Linux distributions (Ubuntu, Debian, Fedora, OpenSuse, Gentoo, ArchLinux).

Spyder is useful and reliable Python software that features advanced editing, tools for data inspection, interactive testing and debugging. Also it embeds Python-specific code quality assurance and introspection instruments, such as Pyflakes, Pylint and rope. This IDE integrates such tools as NumPy, SciPy, Matplotlib and IPython, as well as other open source software. Spyder is a part of spyderlib, a Python module based on PyQt4, pyflakes, rope and sphinx that provides robust PyQt4 or PySide widgets like source code editors, Python console

or GUI-based dictionary, lists/tuples and NumPy array editors. Spyder offers PYTHON-like PYTHONPATH management dialog box (works with all consoles).

Libraries used for the Image processing in python:

1. OpenCV

OpenCV is an open-source library that was developed by Intel in the year 2000. It is mostly used in computer vision tasks such as object detection, face detection, face recognition, image segmentation, etc but also contains a lot of useful functions that you may need in ML.

2. Scikit-Image

scikit-image is a python-based image processing library that has some parts written in **Cython** (Cython is a programming language which is a superset of Python programming language designed to have performance like C programming language.) to achieve good performance. It includes algorithms for:

- Segmentation,
- Geometric transformations,
- Color space manipulation,
- Analysis,
- Filtering, Morphology,
- Feature detection, and more

3. SciPy

Scipy is used for mathematical and scientific computations but can also perform multi-dimensional image processing using the submodule **scipy.ndimage**. It provides functions to operate on n-dimensional Numpy arrays and at the end of the day images are just that. **Scipy** offers the most commonly used image processing operations like:

- Reading Images
- Image Segmentation
- Convolution

- Face Detection
- Feature Extraction and so on.

4. Pillow/PIL

Python Imaging Library (PIL) is an open-source library for image processing tasks that requires python programming language. **PIL** can perform tasks on an image such as reading, rescaling, saving in different image formats. **PIL** can be used for Image archives, Image processing, Image display.

5. NumPy

An image is essentially an array of pixel values where each pixel is represented by 1 (greyscale) or 3 (RGB) values. Therefore, NumPy can easily perform tasks such as image cropping, masking, or manipulation of pixel values.

6. Mahotas

Mahotas is another image processing and computer vision library that was designed for **bioimage informatics**. It reads and writes images in NumPy array, and is implemented in C++ with a smooth python interface. The most popular functions of Mahotas are

- Watershed
- Convex points calculations
- Morphological Processing
- Template Matching.

CHAPTER 4

SYSTEM IMPLEMENTATION

Lane detection using Convolutional Neural Networks (CNNs) and Long Short-Term Memory (LSTM) is a technique used in autonomous driving systems. CNNs are proficient at extracting spatial features from individual frames, LSTM networks are useful for modeling temporal dependencies and capturing long-range context in sequential data. By combining these two types of neural networks, the model can effectively leverage both spatial and temporal information for accurate lane detection in video streams.

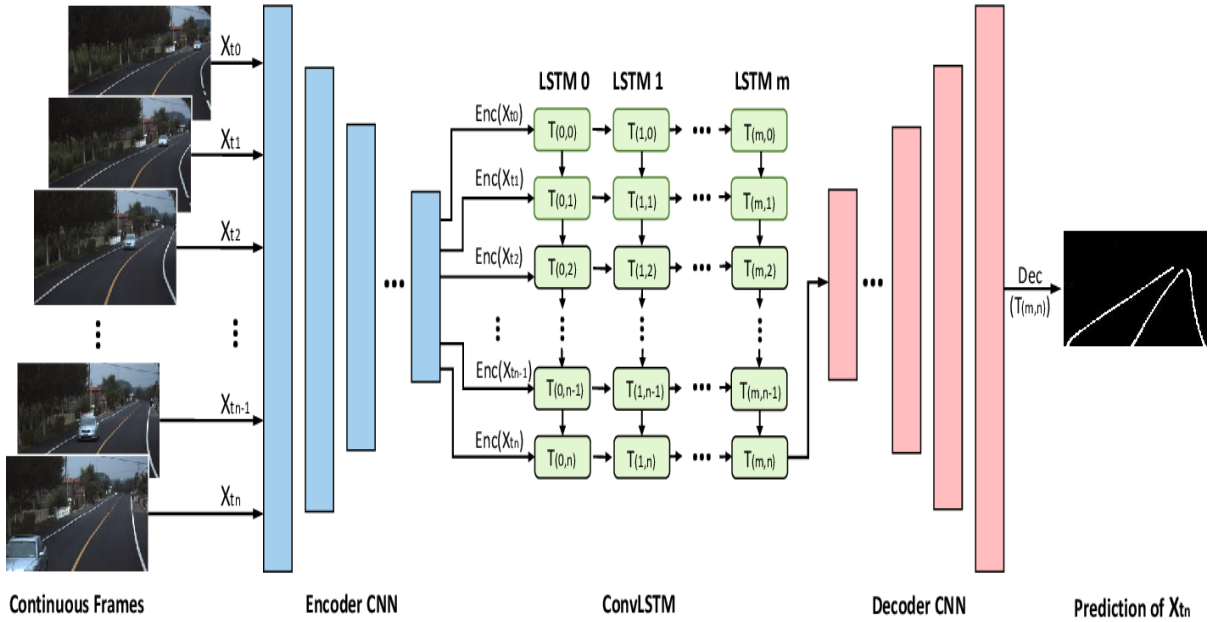


Fig. 4.1. Architecture of ConvLSTM

ConvLSTM, or Convolutional Long Short-Term Memory, is an extension of the traditional LSTM architecture that incorporates convolutional operations. It's commonly used for sequence prediction tasks, particularly when dealing with spatiotemporal data such as videos or image sequences. ConvLSTM in 5 steps:

Step 1. LSTM:

- LSTM (Long Short-Term Memory) is a type of Recurrent Neural Network (RNN) designed to address the vanishing gradient problem in traditional RNNs. It includes

memory cells and gates (input, output, and forget gates) that allow it to selectively remember or forget information over long sequences.

- LSTMs are effective in capturing long-term dependencies in sequential data.

Step 2. Adding Convolutional Operations:

- ConvLSTM extends the LSTM architecture by incorporating convolutional operations into the cell state transitions and gating mechanisms.
- Instead of processing one element at a time, ConvLSTM processes entire spatial regions simultaneously using convolutional filters. This is particularly useful for spatiotemporal data like videos, where both spatial and temporal dependencies are important.

Step 3. Convolutional Gates and Operations:

- ConvLSTM introduces convolutional versions of the input, output, and forget gates. These gates use convolutional filters to capture spatial patterns and relationships within the input sequence.
- The convolutional operations help ConvLSTM learn spatial hierarchies and dependencies, making it well-suited for tasks where both local and global context are essential.

Step 4. Memory Cell Updates:

- Similar to traditional LSTMs, ConvLSTM has a memory cell that stores and updates information over time. The cell state is updated based on the convolutional operations and the gating mechanisms.
- The convolutional operations allow the model to focus on specific spatial features while updating the memory cell, enabling it to capture complex spatiotemporal patterns.

Step 5. Applications in Spatiotemporal Data Processing:

- ConvLSTM is particularly useful for tasks involving spatiotemporal data, such as video analysis, action recognition, and sequence prediction in image sequences.

- It allows the model to simultaneously consider both spatial and temporal dependencies in the input data, making it well-suited for applications where understanding both the spatial layout and temporal evolution of features is crucial.

ConvLSTM enhances traditional LSTM architectures by incorporating convolutional operations, enabling the model to effectively process spatiotemporal data. This makes ConvLSTM a powerful tool for tasks that require understanding both the spatial and temporal aspects of sequential data, such as video processing and analysis.

4.1 LSTM and ConvLSTM for time series data

The Recurrent Neural Network (RNN) was used to process time series data. However, the RNN encountered the vanishing and exploding gradient problems. Subsequently, the LSTM was proposed to model sequence data processing. The LSTM is based on the gating mechanism, which solves the vanishing and exploding gradient problems. The LSTM has been an effective tool for modeling sequential and time series data in recent years. Since then, more studies have used LSTM to build models with encoder and decoder architectures to solve the trajectory prediction problem. When the LSTM is applied to driving intention prediction, the input of the model includes the historical information of the vehicle and environmental information of surrounding vehicles. The information of surrounding vehicles is directly concatenated into the target vehicle as new features of the target vehicle information. This system analyzes the influence of surrounding vehicles from a macroscopic perspective.

The ConvLSTM network was originally used for precipitation now casting. Compared with LSTM, ConvLSTM can model spatiotemporal information simultaneously. In recent years, ConvLSTM has been widely used in spatiotemporal data modeling, such as travel demand forecasting and traffic accident prediction. Inspired by these ideas, this system considers the dynamic change process of surrounding vehicles from a macroscopic perspective, which involves both highly interactive spatial interactions and dynamic change in time. This system proposes an ensemble model, including LSTM and ConvLSTM subnetworks, which process the information of the target and surrounding vehicles, respectively.

4.1.1 Data Extraction and Process

First, a typical driving scenario on an urban road is taken as an example to explain the problem of predicting driving intention. Then, we extracted data from urban driving scenarios and constructed data samples for predicting driving intention.

As shown in Fig. 4.2, the target vehicle can move along the longitudinal direction of the road, such as accelerating, decelerating, and holding. The target vehicle can also move in the lateral direction, such as changing lanes to the left or right. We want to predict its longitudinal and lateral driving intentions simultaneously.

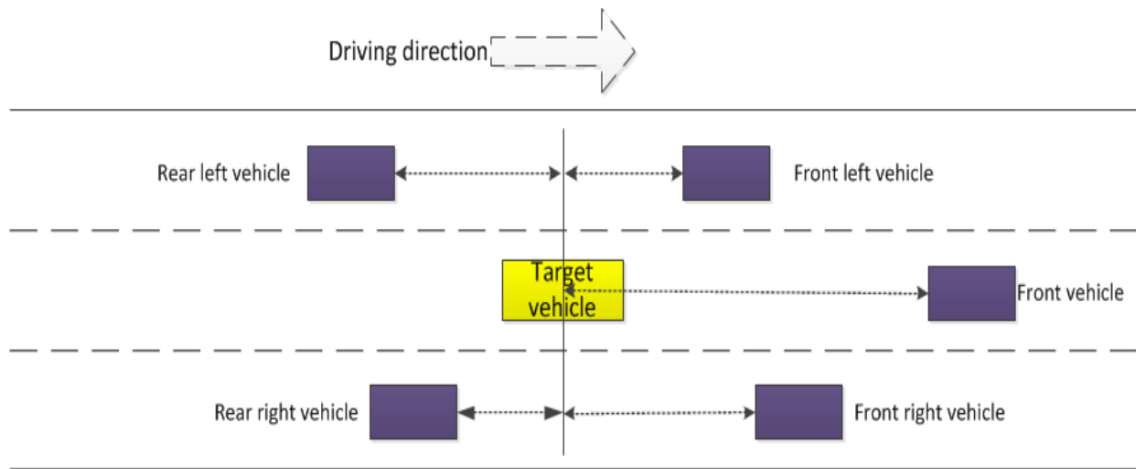


Fig.4.2 A typical urban road driving scenario

To construct a dataset for supervised driving intention prediction, we extracted the required data from the BDD100K dataset with a sampling frequency of 5 Hz.

4.1.2 Data extraction

In the scenario shown in Fig. 4.1, when predicting the future driving intention of the target vehicle, we use its historical motion information to make inferences, although the motion data may not show the typical characteristics. More information, such as the existence of adjacent lanes and relative motion information of surrounding vehicles, has a significant influence on predicting the driving intention of the target vehicle.

The surrounding vehicles have three typical locations, namely, front, left front, and right front. If a vehicle does not exist at a certain location, then the corresponding virtual vehicle is filled in this system, as has been previously reported. In this situation, the relative speed is set to 0, and the relative distance is set to have a large value.

4.1.3 Sample construction for prediction

To predict driving intention, the sample data need to be a historical window with the length of His_t , and this window must be taken before the driving intention starts. The corresponding prediction horizon is Fut_t . The sample construction process shown in Fig. 4.2 has two important points that need to be clarified and explained:

- (1) This system aims to analyze the performance of driving intention prediction under different prediction horizons. Therefore, only one sample is extracted from the file corresponding to each driving intention, rather than constructing multiple samples by sliding windows. Thus, the number and proportion of various types of samples in the complete dataset used for training and testing are the same.
- (2) Two key parameters are used in this system and should be described. The first key parameter is the prediction horizon, which is how early a driving intention can be predicted. The second key parameter is the historical window size, which is how much historical information should be used to make a prediction.

To model the spatiotemporal relationship of surrounding vehicles, the historical motion data of each surrounding vehicle need to be extracted according to the process. First, we divide the left, middle, and right lanes and the front positions into 33 grids, where each column corresponds to a single lane and the rows are separated by a distance of 5 m, which is approximately equal to one car length. Each surrounding vehicle has multiple features at each time step t_i . Then, the motion data of surrounding vehicles at certain time steps are stacked along the channel axis to form a three-dimensional tensor. Finally, the three-dimensional tensors generated at different time steps are stacked into four-dimensional tensors, which are used as the input of the ConvLSTM network.

4.1.4 Input, output, and model architecture

Input : The input for driving intention prediction has two parts, namely, the target vehicle's motion and road constraint information and the relative motion information of surrounding vehicles, as described in Section 3.3. We denote the target vehicle information as Tar info and the surrounding vehicle information as Sur info. Therefore,

$$\begin{aligned} Tar_info &= [x_{t_1}, x_{t_2}, \dots, x_{t_h}]^T, \\ x_{t_i} &= [x^{f_1}, x^{f_2}, \dots, x^{f_m}] \end{aligned} \quad (1)$$

where t_h denotes the window size and f_m denotes all of the features of the target vehicle.

Output: The output is the prediction result of driving intention, including the four longitudinal driving intentions and two lateral driving intentions. Given that the results are formulated as category data, they need to be numerically processed and one-hot encoded as Label Y .

Model architecture: The model needs to process both the historical motion information of the target vehicle and the historical information of surrounding vehicles. To mine the historical information of the target vehicle, the LSTM layer is used for feature extraction. To model the influence of surrounding vehicles, we consider the surrounding vehicles as environmental context. This system proposes a network model based on ConvLSTM, which can learn to capture spatial interactions of surrounding vehicles and their time dependence. The ConvLSTM-based model framework is shown in Fig. 4.2.

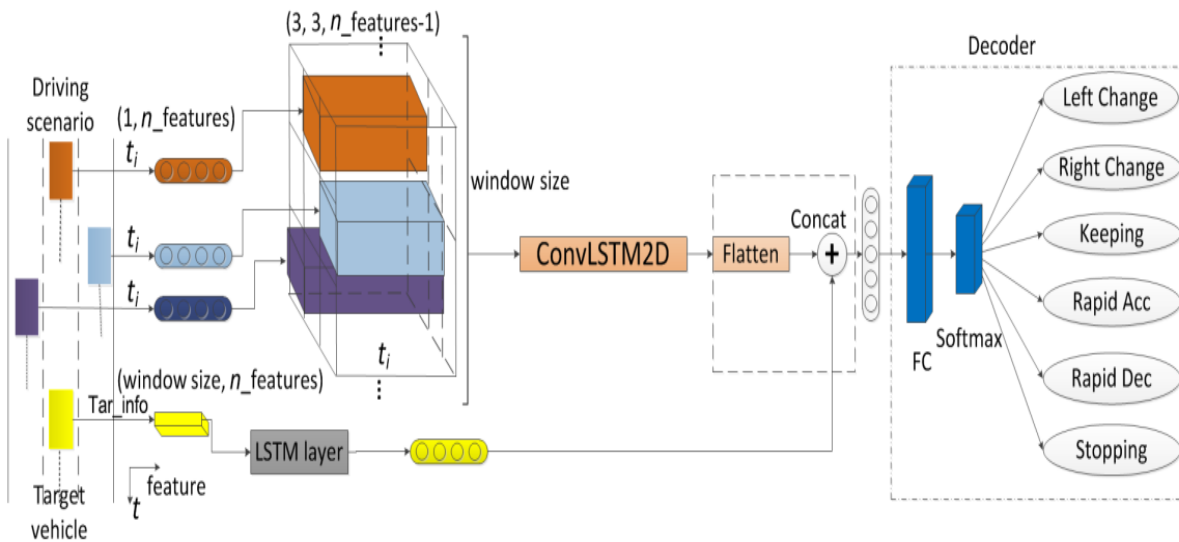


Fig. 4.3 ConvLSTM-based model framework

The model shown in Fig. 4.3 is mainly composed of two parts. The first part is the input layer, which represents the motion information of the target vehicle as a two-dimensional matrix. The historical motion information of the target vehicle is processed through an LSTM encoder with a peephole. According to the grid position occupied by the surrounding vehicles, the data of surrounding vehicles at different time form a three-dimensional tensor, including row positions, column positions, and channels. Three-dimensional tensors at different time steps are stacked together in a time sequence to form a four-dimensional tensor. The ConvLSTM layer is used to capture the spatial interactions of surrounding vehicles. Then, the vector obtained using the ConvLSTM layer and compression

vector of the target vehicle are concatenated together as a final representation of the original input data.

The second part is the output layer. The last fully connected layer uses the softmax activation function to output the probability of each driving intention. Six output neurons correspond to the longitudinal and lateral driving intentions. The neuron with the largest probability value is used as the final prediction result.

4.2 MODULES

The experimental process is based on live and inexpensive camera. Processes of the system is to take video as Input from a Camera.

- Frame Extraction module
- Data Collection and Preprocessing
- Temporal Context Encoding with LSTM
- Lane Detection

Frame Extraction Module:

The Deep learning LSTM method is applied to derive the state of each feature on a frame by frame basis. After providing the video, frame extraction function is running. It reads the video file frame by frame, resizes each frame, normalizes the resized frame, appends the normalized frame to a list, and finally returns the list. Extraction of the dataset is mainly depends on OpenCV via python libraries and tools.

Data Collection and Preprocessing:

This system Gather a dataset of images or video sequences showing road scenes with visible lane markings. The TuSimple dataset consists of 6,408 road images on highways. The resolution of image is 1280×720. The dataset is composed of 3,626 for training, 358 for validation, and 2,782 for testing called the TuSimple test set of which the images are under different weather conditions.

Preprocess the data, which may involve resizing, normalization, and augmentation techniques to improve the model's generalization. The images from the video are in true color. They store the information of Red, Blue and Green (RGB) levels for each pixel in the image. However, it is noted that calculations on RGB is quite time expensive, hence the

Image is converted to grayscale which has only gray level values. The RGB images also use more of disk space and memory. Noise in an image is the consequence of errors in image acquisition process and results in image pixel values that do not reflect the true intensities of the real scene concerned. Noises in an image may be reduced or removed by using a filtering technique. Filtering is a technique of enhancing an image for emphasizing certain features or removing other features. Linear filtering technique is used in this proposed system. In median filtering, the value of pixel in the output image is a linear combination of the values of the pixels in the neighborhood of the corresponding pixel in the input image. These weights in linear combination, is determined with Convolution and Correlation techniques. In an averaging linear filter, the weights assigned to neighbors are equal.

3. Temporal Context Encoding with LSTM:

- Use a CNN architecture to extract meaningful features from each frame of the input video.
- The CNN learns to detect relevant patterns and features such as lane markings, road edges, and other contextual information.
- Convert the output feature maps from the CNN into a sequence of feature vectors.
- Utilize an LSTM network to capture temporal dependencies and contextual information across consecutive frames.
- LSTM helps the model to understand the sequential nature of video data and make predictions based on the history of frames.
- Split the dataset into training, validation, and testing sets.
- Train the combined CNN-LSTM model using supervised learning, where the ground truth annotations for lane markings are used as targets.

4. Lane Detection:

- Feed the sequence of feature vectors into the LSTM module.
- Train the LSTM to predict the position or boundaries of the lane markings in the next frame based on the learned temporal dependencies.
- Output the predicted lane markings or boundaries.

CHAPTER 5

EXPERIMENTAL PROCESS

The computer configuration used in this experiment is Intel Core i5-6200U 11th gen CPU @ 2.40GHz speed, the operating system is Windows 10, and also uses 8 GB memory for accelerate training. The software used for experiment is Python in Jupyter Notebook (Anaconda) tool. The platform used is the tensorflow 6.0 developed by Google, which is mainly used for machine learning and artificial intelligence. The dataset images were collected from Kaggle.com.

5.1 DATASET



Fig. 5.1 Training Images and Test Images from TuSimple Dataset

The TuSimple dataset consists of 6,408 road images on US highways. The resolution of image is 1280×720. The dataset is composed of 3,626 for training, 358 for validation, and 2,782 for testing called the TuSimple test set of which the images are under different weather conditions.

5.2 SCREENSHOTS

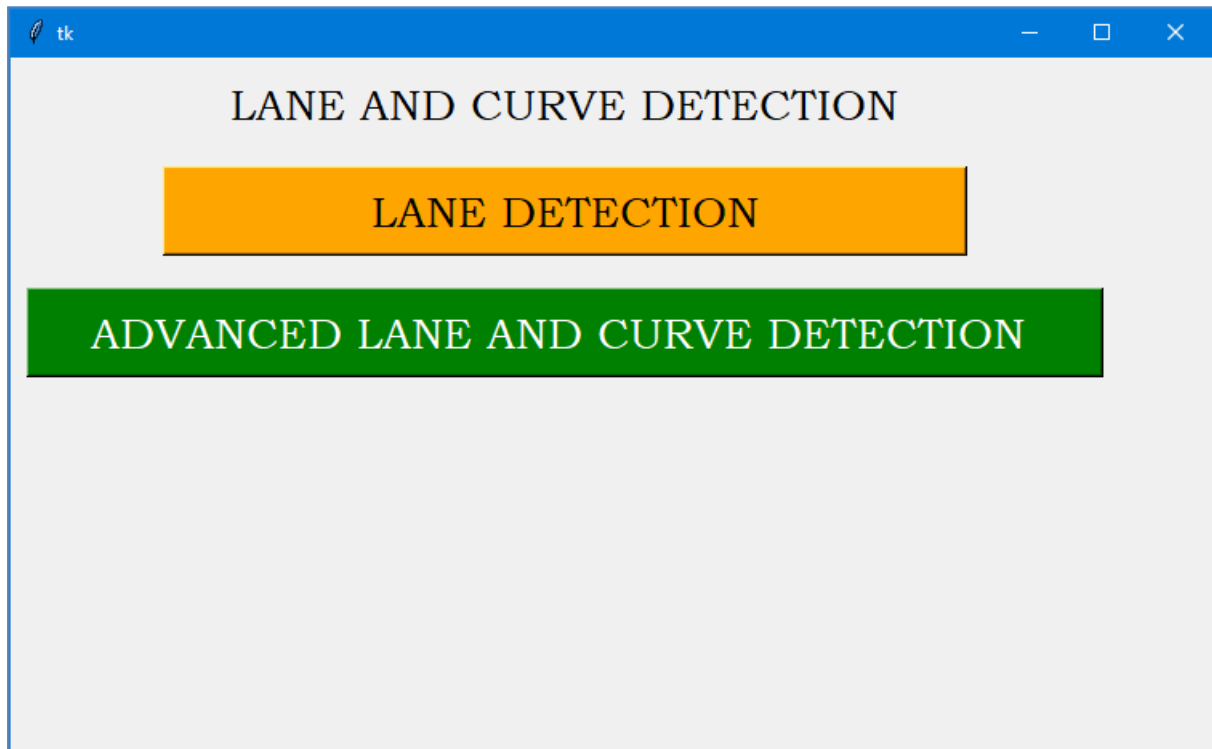


Fig. 5.2 Main GUI window for the project

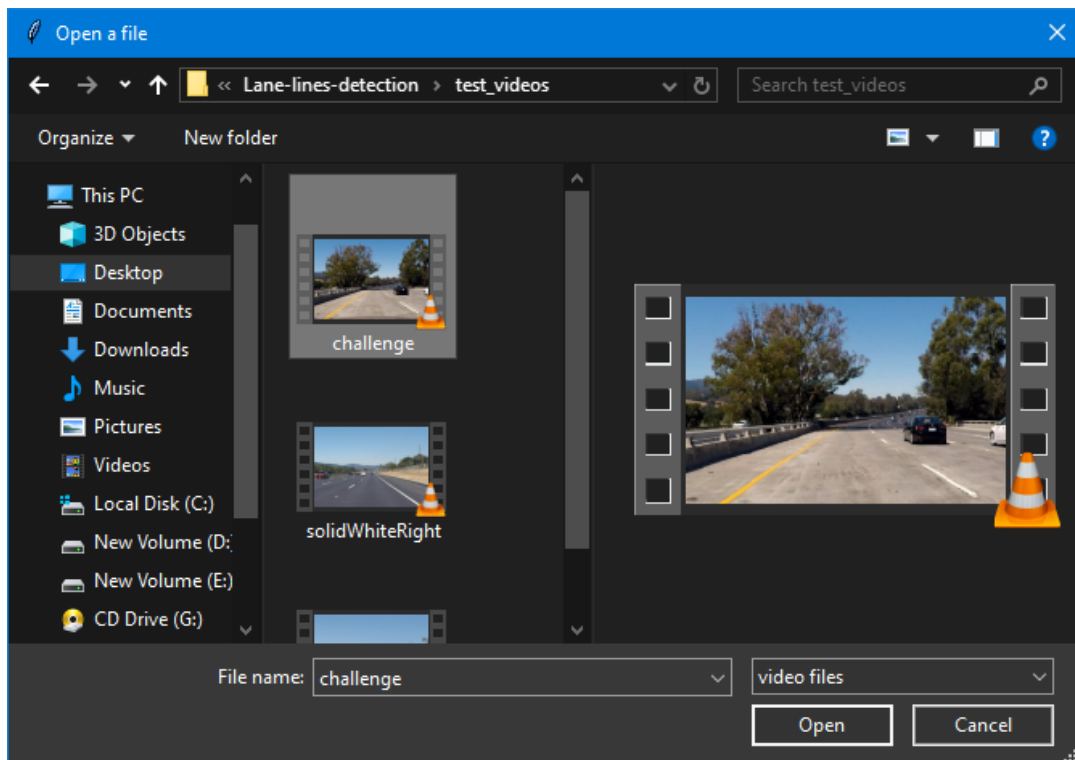


Fig. 5.3 Select the Video file

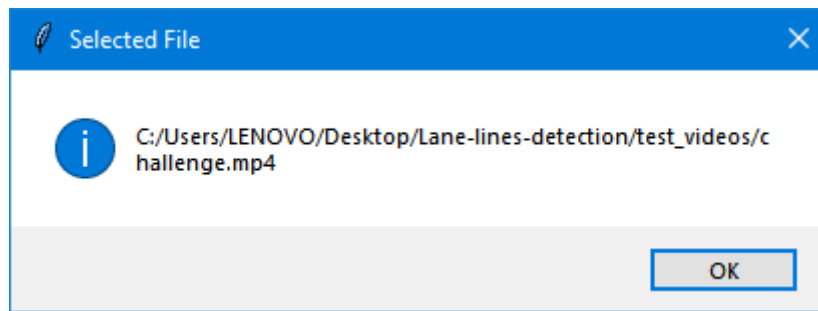


Fig. 5.4 Selected video input

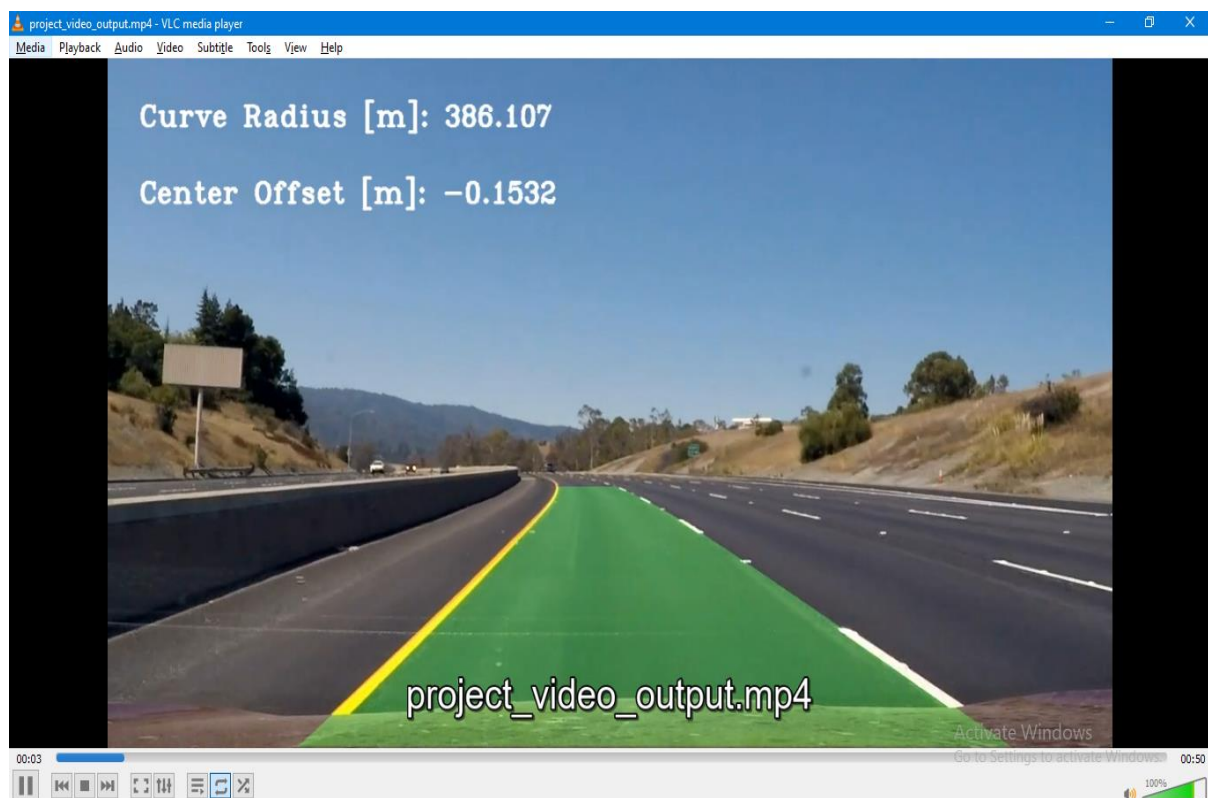


Fig. 5.5 Lane and Curve Detection



Fig. 5.6 Lane and Curve Detection

5.3 PERFORMANCE MEASURES

In order to evaluate the performance of the classifier, and to allow comparisons, several ratios have been taken into account.

True Positive (TP) : Test samples correctly identified.

False Positive (FP) : Test samples incorrectly identified.

True Negative (TN) : Test samples correctly rejected.

False Negative (FN) : Test samples incorrectly rejected.

Precision is a description of a level of measurement that yields consistent results when repeated.

Recall (also called the true positive rate or the sensitivity in some fields) measures the proportion of positives that are correctly identified.

Specificity (also called the true negative rate) measures the proportion of negatives that are correctly identified. The higher the sensitivity and specificity values, the better the procedure. From these quantities, Precision, Recall, Specificity and F-Measure are chosen as performance measures and are calculated using the following equation.

F-Measure: It is harmonic mean of precision and recall.

$$\text{Precision (P)} = 100 \times \frac{TP}{TP + FN}$$

$$\text{Recall (R)} = \frac{TP}{TP + FN}$$

$$\text{Specificity (SP)} = 100 \times \frac{TN}{TN + FP}$$

$$\text{F-Measure (F)} = \frac{2PR}{(P + R)}$$

In order to examine the classification techniques more closely the sensitivity, specificity and accuracy value have been calculated using the confusion matrix shown in Table 5.1.

Table 5.1 Experimental Result

	Precision	Recall	F-Score
Video1	83	100	95.02
Video2	95	82	98.04
Video3	0.85	100	94.91
Video4	100	90	98.00
Accuracy			96.49

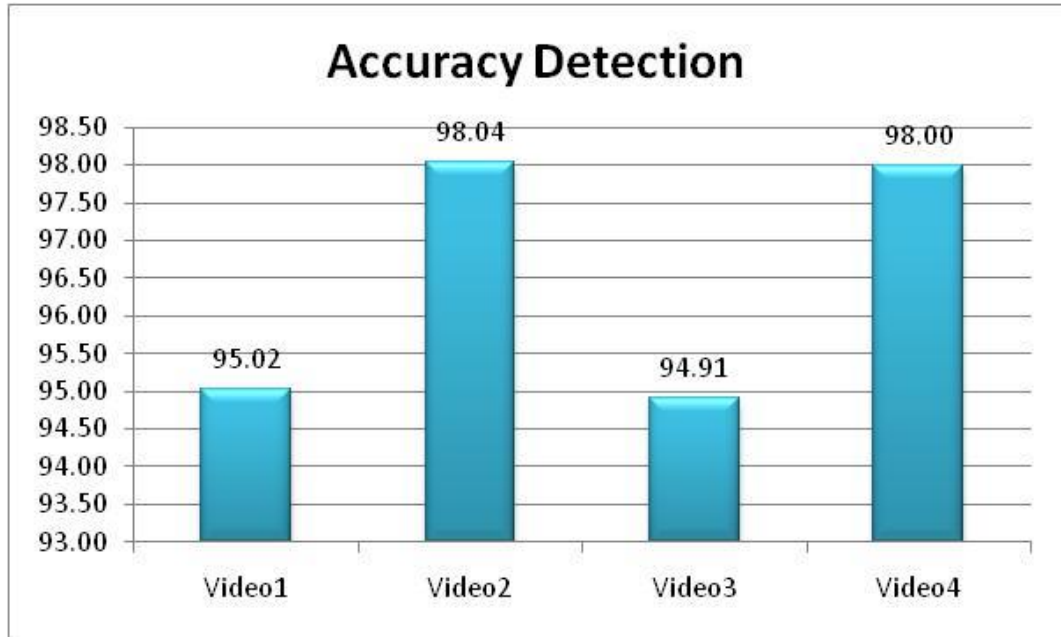


Fig. 5.7 Accuracy comparison

These results can also help us to understand the effectiveness of ConvLSTM. Constrained by its small size, the feature map extracted by the encoder cannot contain all the information of the lanes. So in some extent, the decoder has to imagine to predict the results. When using multiple frames as input, the ConvLSTM integrates feature maps extracted from consecutive images together, and these features enable the model to get more comprehensive and richer lane information, which can help the decoder to make more accurate predictions. frames as an input, and predicts the lane of the current frame in a semantic segmentation manner. In this framework, features on each frame of the input were firstly abstracted by a CNN encoder. Then, the sequential encoded features of all input frames were processed by a ConvLSTM. Finally, outputs of the ConvLSTM were fed into the CNN decoder for information reconstruction and lane prediction. Two datasets containing continuous driving images are constructed for performance evaluation.

CHAPTER 6

CONCLUSION & FUTURE WORK

In this project, a novel hybrid neural network combining CNN and RNN was proposed for robust lane detection in driving scenes. The proposed network architecture was built on an encoder-decoder framework, which takes multiple continuous frames as an input, and predicts the lane of the current frame in a semantic segmentation manner. In this framework, features on each frame of the input were firstly abstracted by a CNN encoder. Then, the sequential encoded features of all input frames were processed by a ConvLSTM. Finally, outputs of the ConvLSTM were fed into the CNN decoder for information reconstruction and lane prediction. Two datasets containing continuous driving images are constructed for performance evaluation. Compared with other models, the proposed models showed higher performance with relatively higher precision, recall, and accuracy values. In addition, the proposed models were tested on a dataset with very challenging driving scenes to check the robustness. The results showed that the proposed models can stably detect the lanes in diverse situations and can well avoid false recognitions. It achieved highest 98.4% accuracy and the overall average accuracy for the tested videos is 96.49%.

In the future, it can be planned to further enhance the lane detection system by adding lane fitting into the proposed framework. In this way, the detected lanes will be smoother and have better integrity. Besides, when strong interferences exist in dim environment, SegNet-ConvLSTM was found to function better than UNet-ConvLSTM, which needs more investigations.

REFERENCES

- 1) Zillur Rahman and Brendan Tran Morris, 2023, LVLane: Deep Learning for Lane Detection and Classification in Challenging Conditions, arXiv:2307.06853v2 [cs.CV] 18 Aug 2023
- 2) Qin Zou, Hanwen Jiang, Qiyu Dai, Yuanhao Yue, Long Chen, and Qian Wang, 2020, Robust Lane Detection from Continuous Driving Scenes Using Deep Neural Networks, arXiv:1903.02193v2 [cs.CV] 29 Apr 2020
- 3) Manoj Kumar Sharma, Ashish Malik, and Aditya Narayan, Lane Detection Using Convolutional Neural Network And Recurrent Neural Network For Autonomous Vehicles, RESEARCH ARTICLE | JUNE 02 2023
- 4) Rustam Abdrakhmanov, Madina Elemesova, Botagoz Zhussipbek, Indira Bainazarova, Tursinbay Turymbetov, Zhalgas Mendibayev, 2023, Mask R-CNN Approach to Real-Time Lane Detection for Autonomous Vehicles, (IJACSA) International Journal of Advanced Computer Science and Applications, Vol. 14, No. 5, 2023
- 5) Fareha Tabassum, Rumana Jawereya, H. Syed Arshiya Salma, Mohammed Abdul Bari, 2023, Road Lane Line Detection System by Using Cnn and Rnn Algorithms in Deep Learning, Mathematical Statistician and Engineering Applications, Page Number: 1345-1352 Publication Issue: Vol. 72 No. 1 (2023)
- 6) Raviteja Kanakala V, Jagan Mohan K, Krishna Reddy V, (2023), Modelling a deep network using CNN and RNN for accident classification, Measurement: Sensors, Volume 27, 2023, 100794, ISSN 2665-9174
- 7) A. A. Assidiq, O. O. Khalifa, R. Islam, and S. Khan, "Real time lane detection for autonomous vehicles," in Computer and Communication Engineering, 2008. ICCCE 2008. International Conference on, 2008, pp. 82–88.
- 8) H. Li and F. Nashashibi, "Lane Detection (Part I): Mono-Vision Based Method," 2013.
- 9) S. Zhou, Y. Jiang, J. Xi, J. Gong, G. Xiong, and H. Chen, "A novel lane detection based on geometrical model and gabor filter," in Intelligent Vehicles Symposium (IV), 2010 IEEE, 2010, pp. 59–64.
- 10) H. Yu, W. Liu, S. Zhang, H. Yuan, and H. Zhao, "Moving Object Detection Using an In-Vehicle Fish-Eye Camera," in Wireless Communications Networking and Mobile Computing (WiCOM), 2010 6th International Conference on, 2010, pp. 1–6.